GHOST: Hebbian-inspired Scoring Mechanisms for Neural Turnover in LSTMs

Ziyaad Shaboodien
University of Cape Town
Cape Town, South Africa
SHBZIY001@myuct.ac.za

Abstract

A significant challenge in neural network design is continual learning: the ability to learn new tasks without catastrophically forgetting previously learned ones. Complementary Turnover Training (CTT) introduces a biologically-inspired architecture over LSTMs, where neurons are scored on scales of redundancy and importance, killed and reborn, slowly being reintroduced to the network over time. We propose GHOST (Gradient-free Hebbian Observation and Selective Turnover), a Hebbian-inspired scoring criterion, and evaluate it against the original CTT scoring and three baselines in a reinforcement-learning foraging task whose rewards reverse across four sequential seasons. Across ten seeds, the Hebbian-inspired criterion is statistically indistinguishable from the original importance-redundancy scoring on every metric, indicating that the scoring criterion is not the source of CTT's benefit. CTT architecture models did, however, retain significantly more of their learned behaviour than standard LSTMs. Crucially, no method, including combined turnover, weight consolidation, and replay, prevented the complete forgetting of the earliest season. For biologically-inspired agent design, this suggests future effort is better directed at the non-stationary learning setting than at refining neuron-selection criteria.

1 Introduction

Long Short-Term Memory (LSTM) neural networks, at the time of inception, were a noteworthy stride in managing memory in the slightly broader field of recurrent neural networks. Its application is primarily in sequential tasks, wherein over time, each cell (hereafter: "neuron") uses gating mechanisms to determine what is strategically best to "remember" over the sequence for optimising its task efficiency. Below, we discuss the main problem still facing LSTMs despite their memory / temporal management: catastrophic forgetting. We then elaborate on proposed strategies against this, motivating our research question for this paper.

1.1 Catastrophic Forgetting

LSTMs factor computation through the generation of a cell state, c_t, and a hidden state, h_t, as a function of the previous cell and hidden state values and the current timestep's input. [9] This is generally used for language-based tasks such as translation or sentence-level word generation. These often require some level of labelled-data training, allowing LSTMs to reach their full potential of memory management and their autolearning mechanisms.

LSTMs, however, are prone to catastrophic forgetting, and are generally weak against continual learning. This is to say that while they are good at managing a single long-term sequential task, they still suffer from being able to manage multiple sequential tasks. Mechanisms have since been introduced to improve generalisation in neural networks, some of which have been applied in architectures to mitigate against catastrophic forgetting. Dropout [24], during training, randomly sets a fraction of neuron activations to zero on each forward pass, preventing units from co-adapting and thereby reducing overfitting. Zoneout [14] is one adapted specifically for recurrent networks like LSTMs: rather than zeroing activations, it randomly forces a subset of hidden units to retain their value from the previous timestep, regularising the network while preserving information across time. Both mechanisms are stochastic regularisation techniques applied per-step during training, and rely on random selection of which units to affect as opposed to any criterion of which units are worth preserving. These have had specific algorithmic improvements [1, 20, 32] in recent years, but the reliance on randomness still remains.

There are limitations to this however. Oftentimes, the weights still collectively gear towards the new task even though there are now zeroed out weights that theoretically should prevent overfitting, or allow for some regularisation in the network. This has forced people developing mechanisms outside of just neuron scoring and pure regularisation to counter overall catastrophic forgetting, including replay, and continual-learning-specific regularisation techniques.

1.2 Complementary Turnover Training

In the original code[1], it introduces a mechanism to test against this: Complementary Turnover Training (CTT), whose design echoes the complementary learning systems view of hippocampal-neocortical consolidation [17]. CTT acts as a five-step process for introducing both a specific scoring algorithm as a counter to the randomness of dropout and zoneout, as well as other biological mechanisms shown in literature to model well against catastrophic forgetting. CTT works as follows:

(1) Neurons in the system are scored after a certain amount of steps. This score is based on the redundancy and importance of neurons, elaborated further in section 3.2.
(2) Thereafter, the bottom p% of neurons are permanently removed, while the top k% of neurons are protected. There must be some minimum x% of neurons still alive within the system during this process.
(3) Replace the killed neurons with either a random small integer, or clone and mutate from the high-importance survivors.
(4) The newborn neurons are then slowly introduced through a gating mechanism to prevent sudden destabilisation of the network.

[1] https://anonymous.4open.science/r/targeted-neuron-turnover/README.md

(5) Finally, it undergoes homeostatic regularisation [31] so as to ensure that the neural network is not degraded at any point during turnover. To counter against catastrophic forgetting, a replay buffer is used during turnover events.

1.3 Hebbian Rules and Research Aims

This paper will do two novel comparisons simultaneously. Firstly, this paper will be measuring CTT as a mechanism and sequence of specific processes to counter against catastrophic forgetting in a reinforcement learning environment. This is important for two reasons:

(1) In general, LSTMs trained via reinforcement learning, a goal-oriented learning driven by reward as opposed to labelled targets, are far less studied than LSTMs on supervised task setting such as sentiment analysis or translation [15, 27]. Literature on LSTMs in reinforcement learning settings is comparatively sparse, and this paper aims to contribute to that gap.
(2) Given CTT is a newly developed process, it has not been explored in a reinforcement learning setting. This paper will serve to benchmark CTT's performance in that context.

The second comparison this will be measuring is specifically within the scoring system. This paper will specifically compare the performance of CTT against catastrophic forgetting by using two separate scoring algorithms: the original CTT one (Equation 2), and our proposed *scoring* algorithm: GHOST, or Gradient-free Hebbian Observation and Selective Turnover (Equation 5). The original CTT scoring system measures neurons based on the importance and redundancy of the neuron activations within the network by measuring the impact of the neuron within the network gradients. GHOST is instead inspired by Hebbian Learning [8]. Hebbian Learning is known as a weight update rule, with the famous tagline of "neurons that fire together, wire together", mathematically represented as $\Delta w_{ij} \propto x_i y_j$. GHOST adapts this idea into a scoring mechanism by measuring importance as a neuron's coactivation with its input. GHOST goes one step further, inspired by the biological theory of the synaptic homeostasis hypothesis (SHY) to do turnover during a "sleep" period of the model, mimicking SHY's proposal that the human brain changes the weights of its own neurons during human sleep periods. [28] This is further important for two reasons:

(1) To our knowledge, no prior work uses a Hebbian-inspired scoring mechanism, thus marking a novel contribution to the field of artificial intelligence
(2) Generally in fields of neuroscience, having some computational modelling of theories acts as an assistance. This will therefore act as an interpretation and computational representation of SHY.

The research question of this paper is therefore: Does Hebbian-scored neural turnover during simulated sleep cycles (where a sleep cycle is considered downtime during which we calculate Hebbian-scores for all neurons) enable more robust long-term and short-term memory consolidation in an LSTM foraging agent in comparison to scoring neurons in isolation through CTT?

Thus, this paper has a very specific scope. It is testing specifically continual learning in a reinforcement learning environment. Specifically, this is an adversarial reward-reversal non-stationarity environment and task setting, which is more demanding than the task-switching most continual learning benchmarks use.

There are two testable hypotheses here:

(1) H1: GHOST's Hebbian-inspired coactivation scoring produces greater retention of earlier seasons than the original CTT scoring. This will be measured via retention metrics (outlined in Section 4.1), though namely survivability and forgetting rates.
(2) H2: The CTT architecture, in a reinforcement learning continuous task setting, regardless of the scorer, retains more learned behaviour across seasons than the three other baselines. This will be measured via retention metrics and survivability, as well as mid-season final performance.

2 Related Work

Below we outline the argument for this paper: starting with elaborating on the continual learning problem, then explaining current methods to counter against this, and finally elaborating on Hebbian Learning. We then discuss gaps in literature and the subsequent direction this paper aims to go into.

2.1 The Continual Learning Problem

Neural networks have been subject for quite some time to experiments on mitigating catastrophic forgetting, sometimes known as catastrophic interference [13]. In the context of connectionist networks, which is to say neural networks with weights connecting the nodes within the network, they suffer from continual learning, or what McCloskey and Cohen call the sequential learning problem [18]. The baseline idea is quite simply that even though neural networks are able to perform well in certain tasks, as soon as a new task is learned, the network has already forgotten how to perform its previously learned task [6, 18]. French describes this in more detail by analysing the "weight-space", which is the landscape of all possible weight configurations. This is extended to 3D space, in which the optimality of certain weights often appear as a "peak" in the weight-space [6]. French then points out that catastrophic forgetting can be visualised as the sensitivity between these peaks and valleys: that a weight moving "even small distances over the weight-landscape would radically disrupt prior learning." [6].

2.2 Continual Learning Approaches

There are three main techniques to counter catastrophic forgetting discussed below.

2.2.1 ***Regularisation-Based Techniques.*** The most common technique is Kirkpatrick et al.'s Elastic Weight Consolidation (EWC). When the weights of a network are tuned towards Task A, it applies a quadratic penalty to weight movement away from the solution to Task A when it learns task B. [12]. If there are multiple tasks, EWC must be applied over multiple levels (i.e. one instance of EWC is not enough, it must be applied per task). [12]. Huszár argues that this multiple applied penalty might apply penalties inconsistently to weights between tasks, and proposed a derived EWC laplace algorithm to counter this effect [10]. This has generally showed positive results in mitigating catastrophic forgetting in backpropagation-trained networks [10, 12].

2.2.2 ***Replay-based Techniques.*** Most relevant to this paper is Rolnick's Continual Learning with Experience And Replay (CLEAR). While many others exist, such as iCaRL, GEM, Deep Generative Replay, etc. Rolnick proposes an agent undergoes two techniques simultaneously. The first is an off policy learning, where the agent keeps a buffer of past experience and learns from it off-policy, while using behavioural cloning between its current policy and its past self to preserve old-task behaviour. [21]. The second is an on-policy learning, where the agent simultaneously learns on-policy behaviour from fresh experience so it can adapt and perform toward the current task.[21]. This is most similar to the CTT code we will be using in this paper, as it is one of the few continual learning techniques applied in a reinforcement learning environment, as CLEAR showed similar cumulative performance towards networks trained on the same tasks separately and simultaneously [21].

2.2.3 ***Architecture-based Techniques.*** While architectures like PathNet [5] or Dynamically Expandable Networks [35] have made great strides as being architectural models designed around minimising catastrophic forgetting, most relevant to the biologically-inspired mechanisms, which is where CTT as an architecture sits in the domain of, is Draelos et al.'s Neurogenesis Deep Learning (NDL) [4], as well as Gurbuz et al.'s Neurogenesis Inspired Contextual Encoding [7]. Both are inspired by adult neurogenesis, which has been shown to act as a neural regulariser [30]. NDL adds capacity (which is the birth of neurons) triggered by a reconstruction error. The idea behind NDL is that adding more neurons (as opposed to dropout's technique of zeroing out existing neurons, or zoneout's technique of preserving previous activations) acts as an architectural buffer to ensure that the new neurons are more likely to learn the new task, thereby being more protective of weight movements attuned to the previous task [4]. NICE on the other hand groups neurons together based on their different maturation stages, and infers which neurons to use during testing without requiring additional signals. NICE, on 6 datasets and 3 architectures, performs on par or outperforms replay mechanisms, acting as an architecture that could be used replay-free. NICE shows most importantly as a justification for the GHOST scoring algorithm that neurons specialise within a task, and that activations are capable of being used to infer these specialisations.[7] This is a strong motivation for developing a specific scoring algorithm.

2.3 Hebbian Learning in Neural Networks

Hebbian learning is a general psychological paradigm that is widely adopted as the general model for long-term memory in humans [16, 26]. More abstractly, it describes a specific relationship in the brain wherein we can imagine an input mapped to a specific output is formed specifically through the strengthening of synaptic connections. [26]. It is of great interest to neuroscientists to represent proposed biological models computationally, given the control one has over the simulation and variables being tested. Further, in trying to replicate the computational efficiency of the human brain, there may be some notable improvement to machine computational efficiency [11]. Thus, Hebbian learning as a paradigm has been implemented as a general learning rule within deep neural networks [3, 11]. This has been used as a loss function [3] and as its own learning rule in replacement of backpropagation [11]. This is generally considered apt for more biologically sound representation within neural networks [11], as well as some resistance to noisy inputs while maintaining accuracy [3].

2.4 Gaps in Literature

It becomes quite evident that methods against catastrophic forgetting in the context of reinforcement learning is a relatively understudied field, wherein most studies tend to focus on more labelled data tasks. While CLEAR shows evident adaptability to catastrophic forgetting, it hasn't been studied as an application through a specific continual learning architecture. CTT acts as an architecture combining three of the more prominent ideas in literature, something that hasn't been done in prior works: biologically-inspired mechanisms, introducing neurogenesis in conjunction with neuron death. It also incorporates regularisation through its homeostasis pipeline, as well as replay mechanisms. As an architecture, CTT is entirely novel in the field of continual learning. However, in both the field of continual learning and CTT as an architecture, multiple task sequencing (i.e. more than 2 tasks) is something relatively understudied in terms of strategies to counter catastrophic forgetting.

Thus, with the introduction of GHOST, the aim of this paper is to address the gaps mentioned in the following ways:

(1) Test a proposed architecture of continual learning in multiple sequences of tasks
(2) Have these proposed architectures be tested in a reinforcement learning environment
(3) Measure the specific usefulness of scoring criteria and its specific effect alongside existing continual learning techniques, and measure the difference in performance between them.

3 Experimental Methodology

Below we discuss the overall design elements taken into consideration for this project. In this section, we cover how the environment and task sequence was defined, followed by the models used in the experiments of this paper, and finally an explanation of how the models were trained and evaluated.[2]

3.1 Environment Design

While commonplace to use existing or pre-built reinforcement learning environments, this specific task set is incredibly understudied and thus difficult to find environments online. We therefore had to use the Gymnasium library to design the environment all models would be trained on [29]. The challenge of environmental design is ultimately balancing factors at tension with each other: it can't be an environment that lends itself to easy learning, but cannot be so difficult that sequential learning becomes a futile task. Thus, the final design of the environment is described below.

The environment is broadly a sequence of foraging tasks. It mimics an open field, in which the environment is a 20x20 grid, i.e. 400 total cells, with the agent spawned in the middle of this field. Randomly scattered across the field are 12 patches of food. These numbers were chosen to ensure the environment was sparse enough to promote exploration, but still dense enough that the

[2]All code and results can be found at https://gitlab.cs.uct.ac.za/shbziy001/bionn

agent can reasonably survive every season. The agent has a certain amount of energy in the environment per episode, and thus must replenish it by finding and eating these patches of food in order to regain some energy, thus allowing it to survive the full episode. Not all patches of food are safe to eat, however, and once a patch is eaten, it takes a certain amount of time to "regrow". Finally, the environment undergoes a "year" in the form of seasons, that being summer, autumn, winter, and spring, which lasts 800 episodes each. Thus, the entire experiment lasts 3200 episodes. Notably, each episode, the agent is expected to survive 2000 total steps for it to be deemed successful. Theoretically then, the agent will have a maximum of 6,400,000 steps per simulation, making it a large enough scope to measure its survivability while giving it more than enough time to learn the environment fully. Moreover, having a 2000-step survival threshold across an 800-episode season (or task) length gives the agent enough time to converge on a survival policy within each season while keeping the full task set simulation computationally tractable.

3.1.1 ***Patch Logic.*** Every 3 patches in the environment are directly correlated to a season within the environment. Thus, its task in every season is to find the most optimal paths towards surviving. This optimality is measured twofold: energy yield, and reward yield. Not all patches are necessarily equal. For every 3 patches, they have a slightly increasing yield between them. This is to say that one patch may have a yield of 3.25, while another will have a yield of 3.55, and another 3.85. This variance was designed so that not every in-season patch is equivalent to one another, encouraging the agent to differentiate between patches and develop a more structured or optimal foraging route rather than defaulting to the nearest patch. If a patch of food is associated with the current season of the environment, the agent is given a proportionally positive reward along with its relevant replenished energy. If a patch of food is associated with an adjacent season (i.e. if the current season is summer, then adjacent seasons are autumn and spring), the agent gets no energy from it, and gets a negative reward from it. If the food is completely out of season (i.e. if it's summer, out of season is winter), then the food is labelled "toxic" and is an instant-death for the agent if it eats it. The seasons within the environment are all one-hot coded and observable to the agent. These patches of food are allocated randomly across the field once per experiment simulation, thus forcing the agent to explore and become familiar with the entire environment, and, through later discussed mechanisms, allow the agent to become more deterministic on its path per season. Finally, the last bit of patch logic is its capacity to restore itself. Every single patch has a time to regenerate relative to its seasonality. If in season, it grows back after 40 steps. If in an adjacent season, it grows back after 100 steps. If off-season, it doesn't really matter since it's instant death, but we settled on 300. The first two values were decided upon after sweep testing with an LSTM baseline: any higher than these values each, and the baseline significantly degraded in its learning ability.

3.1.2 ***Energy Logic.*** The environment undergoes two major cycles: a 'mild' cycle, and a 'harsh' cycle. The mild cycle will always occur in the first half of the season, providing the agent with an energy baseline of 50.0. This is then lowered in the 'harsh' cycle, i.e. the second half of the season, to 40.0. However, the agent is negatively rewarded for every action by a constant factor of -0.1, regardless of which cycle the season is in. This is to encourage some level of exploration in the first half, and force more optimisation in the second half, such that the weights of the network by the end of the season are theoretically optimised for surviving in that season indefinitely.

Overall, the following parameters exist within the environment. We can see the patch logic in terms of how much yield each patch has within the environment through Table 1 below:

Table 1: Table showing differences between in-season and off-season patch yields

Condition	Yield Level	Yield Value
Off-season	Low	2.10
Off-season	Med	2.15
Off-season	High	2.20
In-season	Low	3.25
In-season	Med	3.55
In-season	High	3.85

The in-season yield is calculated as 6.5% of the agent's 'mild' energy level, which was decided upon after some preliminary testing. This was generally shown to be enough of a yield to encourage task optimisation. After some preliminary testing, we found that even if the agent had a small energy yield from off-season patches, it would still include them in its path even with a negative reward. Removing any energy drawbacks from the yield showed more positive reinforcement in the agent to optimise towards in-season eating.

3.1.3 ***Wall Logic.*** For every seed, there is a random amount of walls placed throughout the environment. It is set to be between 4 and 7 walls in the environment, of lengths 5-8 cells each. These notably do not intersect the spawn point of the agent, which is (12,12), and anyplace where patches exist in the environment. This is to add difficulty to each task set, and further encourage the agent to explore. If the agent hits a wall, on top of the existing negative reward of an action, it receives an additional negative reward of -0.15. This encourages the agent to explore beyond walls.

3.1.4 ***Exploration Logic.*** Beyond the above general mechanisms that, in theory, allow for the agent to explore, there is one additional mechanism to reward exploration: curiosity, where the algorithm below is inspired. [2, 25] This is calculated in Equation 1

$$r_{\text{int}}(x, y) = \frac{\beta_{\text{cur}}}{\sqrt{N(x, y)}} \cdot \max\left(0.1,\ 1.0 - \frac{t}{1600}\right), \tag{1}$$

where β_{cur} is the curiosity coefficient, $N(x, y)$ is the visit count for cell (x, y), and t is the current episode. The first term rewards visiting rarely-seen cells (decaying as $1/\sqrt{N}$ with repeated visits),while the second term linearly anneals the curiosity bonus over the first 1600 episodes, floored at 0.1 so a minimum level of exploration is retained. The reason why this is desirable is twofold. Firstly, if we are to test continuous memory, it is worth seeing if this early exploration results in some level of determinism later on by virtue of the agent "remembering" visited nodes early on, and applying its usefulness later on. Secondly, if the agent is continuously rewarded

highly to explore, there is an increased risk that the agent becomes slower at being deterministic, which would make it more difficult to measure how good the model is at continual learning.

3.2 Models

There are 5 different types of LSTM models used within this paper: three baselines, and two novel comparatives. All models have two hidden layers, of size 128 neurons each. The input dimension for all models is 105. The output layer is a set of five, based on the possible actions for the model: {stay in place, move up, move down, move left, move right}. The first model is an LSTM with absolutely no neural turnover, dubbed "Base LSTM". The second model is an LSTM with a dropout mechanism, dubbed "Dropout." The third model is an LSTM with zoneout, dubbed "Zoneout".
Given that this paper is aimed at testing the validity of biologically plausible mechanisms as a means of continual learning, these act as our baselines primarily as a comparative metric for whether or not the biologically plausible mechanisms are better at "remembering" multiple tasks.

The final two models both have a CTT architecture. The general structure of a CTT model is to have a non-plastic hidden layer that feeds into a plastic hidden layer. The plastic hidden layer undergoes a turnover process. During the lifetime of the agent, the neurons of the plastic layer are "scored" by an algorithm that measures the importance and redundancy of neurons. Through this algorithm, some top $k\%$ of neurons are protected, while a certain $n\%$ of neurons are then randomly selected to be zeroed out. This is a similar mechanism to zoneout and dropout, though with far less randomisation involved. It undergoes a gating and homeostasis mechanism thereafter.

The original CTT code assumes a supervised setting with labelled targets. Our environment comparatively is reinforcement learning-based: while the agent receives a structured observation at each step, such as its current position, the one-hot encoding of the seasons, and any patch cells or field cells within view, it's never given a target action. It learns purely from the scalar reward signal. Our paper thus provides an EWC and replay mechanism that is adapted specifically for RL tasks.

The comparative models here differ only in how the scoring works. In the model dubbed "CTT_Normal", the scoring system, outlined in equation 2, is looking at the activations from the plastic layer, and measuring their importance in relation to the final output, as well as the redundancy of the activations. It functionally acts as a min-max algorithm: keep the neurons that are important to the final output, but are distinctly different to one another. This is theoretically to ensure the highest breadth of neurons impacting the agent's decision-making capabilities are kept.

$$S(i) \;=\; \alpha\,\mathcal{I}(i) \;-\; \beta\,\mathcal{R}(i), \tag{2}$$

where the importance and redundancy terms are

$$\mathcal{I}(i) = \overline{|a_i|} \;\cdot\; \overline{|g_i|}, \tag{3}$$

$$\mathcal{R}(i) = \max_{j \neq i} \operatorname{corr}\big(h_i,\, h_j\big). \tag{4}$$

Here $S(i)$ is the survival score of neuron i; $\overline{|a_i|}$ and $\overline{|g_i|}$ are exponential moving averages of the magnitudes of neuron i's activation and gradient, respectively; $\operatorname{corr}(h_i, h_j)$ is the Pearson correlation between the activations of neurons i and j; and $\alpha, \beta \geq 0$ weight the importance and redundancy terms. A neuron scores high when it is both active-and-influential ($\mathcal{I}$ in equation 3 must be large) and non-redundant with other neurons ($\mathcal{R}$ in equation 4 must be small); low-scoring neurons are selected for turnover.

The criticism we propose to the original CTT algorithm is that it might prioritise keeping weights only from the plastic layer that don't necessarily connect to the larger network in impacting the final output decision. The second model, which uses our GHOST algorithm (equation 5), changes the CTT pipeline slightly. Firstly, on scoring: GHOST states that for each plastic-layer neuron i, we define its importance as a weighted coactivation between the neuron's activation and the LSTM input features it is connected to:

$$G(i) = \frac{1}{\sum_j |W_{ij}| + \epsilon} \sum_j |W_{ij}| \;\operatorname{corr}\big(x_j,\; a_i\big) \tag{5}$$

where W_{ij} is the plastic-layer weight from LSTM input feature j to plastic neuron i (with weight matrix $W \in \mathbb{R}^{P \times D}$ for P plastic neurons and D input features), x_j is input feature j, a_i is the activation of plastic neuron i, and ϵ is a small constant for numerical stability. The term $\operatorname{corr}(x_j, a_i)$ is the Pearson correlation between input feature j and neuron i's activation, computed over a waking cycle. The leading normalisation (an Oja-style term) [19] divides by the neuron's total incoming weight magnitude so that importance reflects structured responsiveness rather than raw wiring magnitude. We also propose a change to the CTT pipeline. Whereas with the original CTT mechanism turnover happens every n steps (which, after tuning in table 3, $n = 10000$), we propose that turnover ought happen during a simulated "sleep" cycle so as to be more biologically plausible. We undergo sleep every 50 episodes, in which a running storage of all inputs and activations thus far are then evaluated and scored according to equation 5. Thus, the final architecture of the CTT_GHOST pipeline is as follows:

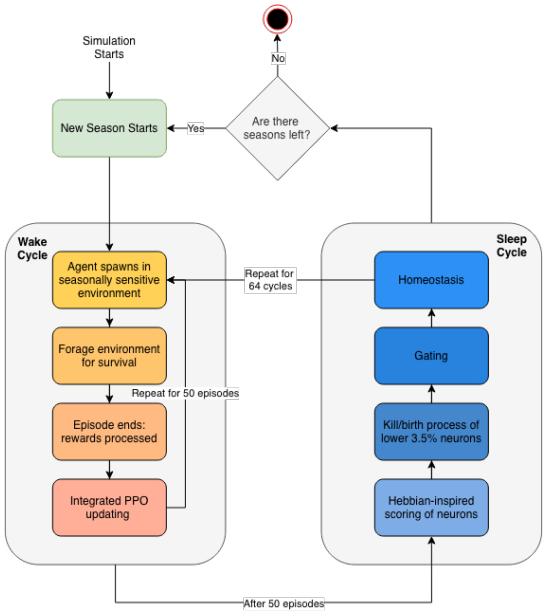


Figure 1: CTT_GHOST agent simulation process

Both the number of episodes for turnover and the bottom killed off percentage were tuned, see table 3 in the section below for further reference.

Thus, in terms of experiments, this paper aims to see if there's credibility to biologically-inspired mechanisms as a valuable counter to catastrophic forgetting in neural networks, while simultaneously measuring if having a Hebbian-inspired scoring mechanism is more statistically better at continual learning than the original CTT scoring mechanism.

3.3 Training and Evaluation

It is important to note that within training, the models all undergo a PPO update with GAE [22, 23] . Thus, for the purposes of hyperparameter tuning, we tuned the base LSTM on specific hyperparameters that would be affected by the PPO update logic, and then used those tuned values when tuning the other models' specific hyperparameters. For all tuning, this was done on a seed value of 27.

For the base LSTM, the values tuned over 60 trials spanning a minimum of 400 episodes, and a maximum of 1000 episodes, can be seen in table 2 below.

Table 2: Hyperparameters tuned for the baseline LSTM with no additional mechanisms

Parameter	Type of Range	Range Tested	Final Value
Learning Rate	Continuous	10^{-5} to 5×10^{-3}	**0.0046**
Entropy Coeff.	Continuous	0.001 to 0.5	**0.0291**
Curiosity Coeff.	Continuous	0.005 to 2.0	**1.3084**
`n_steps`	Discrete	1024, 2048, 4096	**4096**
`n_epochs`	Discrete	2, 4, 6	**2**

Thereafter, for both dropout and zoneout, they were tested in the following discrete range: [0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.4]. This was done over 1200 episodes for each value, where the best performing one was chosen. For dropout, the final value was 0.25. For zoneout the final value was 0.3.

Finally, for the CTT models, this was tuned over five parameters, as seen in Table 3.

Table 3: Hyperparameters tuned for the CTT models

Parameter	Type of Range	Range Tested	Final Value
Death Fraction	Continuous	0.01 to 0.1	**0.03502**
Protect Top K%	Continuous	0.05 to 0.25	**0.1442**
Min Alive Fraction	Continuous	0.6 to 0.85	**0.7284**
Sleep Cycle (`k_steps`)	Discrete	5000, 10000, 18000	**10000**
EWC Lambda	Discrete	100, 1000, 10000	**10000**

Given the original CTT is based on steps, it undergoes replay far more often than the GHOST CTT otherwise would. Thus, we implemented replay 10 times before episode 400, and 20 times after episode 400, for a sleep cycle occurring every 50 episodes.

Each model is trained over 3200 episodes, wherein they are rewarded for surviving 2000 steps an episode. For each season lasting 800 episodes, they are forced to consolidate the best path for survival.

To evaluate each model, the aim is to test, after a full year's training, how well the model retained information from previous seasons. All evaluations are performed on the final frozen weights, run over 200 episodes per season, and averaged across ten seeds.[3] Thus, we evaluate the model in the following ways

(1) To evaluate task performance, we measure the average final reward value for each model per season.
(2) To evaluate survivability, we measure the percentage of episodes in which the agent reaches the 2000-step survival limit per season.
(3) To evaluate partial survival that the binary survivability metric hides, we measure the median episode duration (steps) per season
(4) To measure forgetting, we store the model's weights at the end of each season's training, called a checkpoint. We compare each season's checkpoint on its own season (its peak "learned" performance) against the final model's performance on that season (its "retained" performance). The drop between them is the forgetting. A gate test first confirms a season was learned, since forgetting only matters for a season the model was able to master. Spring, being the last season trained, is identical to the final model, thus serving as a no-forgetting control.

It is worth emphasising that all ten seeds trained to completion for every model without policy collapse. That is to say no run exhibited the entropy collapse or sustained reward divergence that can otherwise destabilise turnover-based training. All reported results reflect stable, fully-trained agents rather than any partially-converged or degenerate runs. Stability was also monitored via the final loss, where a loss exceeding 1,000,000 was treated as a marker of a destabilised run, and no reported run crossed this threshold.

3.4 Ethical Consideration

This study involved no human or animal subjects, and raises no data-privacy concerns as all experiments are conducted in a fully simulated environment with synthetically generated data. The biggest ethical consideration concerns computational power, where training five model variants across ten seeds alongside hyperparameter tuning carries a non-trivial energy cost. This was mitigated by tuning on shorter episode budgets before committing to full runs. All training was done locally. To support reproducibility, all methods are reported in the paper, the code base has been made available, and all seeds are reported. [4]

4 Results and Discussion

We evaluate all five models along three performance axes: final reward, survivability, and episode duration (steps). This is supplemented by a forgetting analysis that separates what the model learned from what it retained. All results are reported over ten seeds. For each model, we evaluate the final frozen weights on 200

[3]The specific seeds can be found in Appendix B
[4]see Appendix B

episodes of each season, with no further learning or turnover, so that the reported performance reflects directly what the network retained rather than any continual adaptation.

For statistical testing, we use non-parametric methods appropriate to the sample size and to the non-normal, floor-bounded distributions. The floor is defined through the primary metrics of survivability and steps, where survivability has a floor of 0, and steps has a pseudo-floor of 500. The use of pseudo here is purely because this isn't an absolute floor, instead being the expected number of steps if an agent were to not eat anything in season. Where agents are less than 500 steps, it indicates having eaten a toxic patch before 500 steps / starvation. The latter is because 500 steps is the maximum steps an agent takes in an environment without eating any in-season food. If any metrics fall to a negative ratio in forgetfulness, it's because an agent was poisoned, dying early.

For each season and metric, we first apply a Friedman test across all five models to establish whether the models differ at all. Where significant, we apply post-hoc Wilcoxon signed-rank tests for three pre-specified comparisons: CTT_GHOST (The GHOST algorithm) versus CTT_Normal (The original importance-redundancy algorithm), and each CTT variant against the LSTM with no architecture or turnover mechanisms. We do this further with Holm-Bonferroni correction for multiple comparisons. We report effect sizes as the matched-pairs rank-biserial correlation and 95% confidence intervals from bootstrap resampling. Throughout, we distinguish a season model never learned from one it learned and then forgot, since retention is only meaningful for the latter.

4.1 Seasonal Learning Capabilities

Forgetting is only meaningful for a season a model actually mastered. Thus, we first establish which model-season combinations were learned. For each model-season, we store "checkpoints" i.e. the weights of a season immediately after it had finished training. Thereafter, we probe its checkpoint on its own season, giving it a "learned" performance peak before any subsequent training erodes it. We test if this peak significantly exceeds the 500-step starvation reference (a one-sided Wilcoxon signed-rank test against it).

Most models learned summer, autumn, and winter. Dropout notably did not learn the task at all. For LSTM with nothing else (LSTM Base), Zoneout, CTT_Normal and CTT_GHOST, peak performance on summer, autumn, and winter was well above the starvation reference (learned duration 1445-2000 steps, peak survivability 50-90%; gate $p < 0.05$ in all cases), confirming these models learned appropriate and survivable foraging behaviour per season. Dropout, by contrast, failed the learning gate in every season (peak duration 490-654 steps, never significantly above the starvation line; gate $p \geq 0.12$). Its checkpoints could not survive even the season they were trained on. Dropout therefore did not learn the foraging tasks, making its poor final performance reflective of its failure to learn rather than a fault of catastrophic forgetting. We exclude Dropout from all subsequent retention comparisons, since a model cannot forget a task it never attained.

Ultimately, learning strength varied per season. Peak performance was highest in summer (with all learners reaching 80-90% survivability and near-maximal step counts). This lowered in autumn and winter, but was still significant, with survivability settling at 50-90%. This aligns with training dynamics: summer was first trained on a fresh set of neurons, thereby making it strongly acquired. Later seasons were thus learned atop a more progressively constrained network. Spring, the last season to be trained, has checkpoints identical to the final model, serving as a no-forgetting control in further analysis.

While not yet answering our research question, this establishes a precondition: both CTT architectures (and the LSTM and Zoneout baselines) learned summer, autumn, and winter well, so any subsequent difference in retention between the Hebbian-inspired scoring system and the original one can be attributed to forgetting rather than a failure to learn.

4.2 Final Model Performance

Here we examine the performance of each model's final frozen weights on every season, i.e. what the model retains after a full year of training. Note in Table 4 below, reward is mean episodic reward, surviv. is the percentage of episodes reaching the 2000-step limit and steps is median episode duration (≈500 = starvation, 2000 = full survival). Furthermore, where any model falls below the starvation floor of 500 steps, it means the agent had eaten a toxic patch and died early (E.g. Zoneout in autumn and LSTM Base autumn). All models fail summer identically (0% survivability), the signature of complete forgetting of the first-trained season. "*" means significantly greater than LSTM Base (Holm-corrected Wilcoxon, $p < 0.05$). No CTT_GHOST vs. CTT_Normal comparison was significant on any metric or season.

Table 4: Final-model performance over 10 seeds, evaluated on frozen weights across all four seasons. Reward and surviv. are means; steps is the median.

Metric	Model	Summer	Autumn	Winter	Spring
Reward	LSTM Base	−144	−119	−142	66
	Dropout	−157	−151	−151	−153
	Zoneout	−137	−111	−138	35
	CTT_Normal	−152	−111	−66	−5
	CTT_GHOST	−141	−86	−110	102
Surviv. (%)	LSTM Base	0	0	0	80
	Dropout	0	0	0	0
	Zoneout	0	0	0	60
	CTT_Normal	0	20	30	50
	CTT_GHOST	0	30	10	90
Steps	LSTM Base	501	284	501	2001
	Dropout	501	501	501	501
	Zoneout	501	92	501	2001
	CTT_Normal	562	528*	1275*	1713
	CTT_GHOST	501	566*	1267*	2001

No model retained full survival on earlier seasons, but the turnover-based models retained partial survival, while the baselines retained none. As the heatmap figure 2 below shows, final-model survivability was 0% for every model in summer, thus marking the earliest trained season forgotten completely by all approaches. In autumn and winter, however, the two CTT models retained measurable survival, while all three baselines collapsed to 0%. Spring, the most

recently trained season, shows the highest survival on average across all models, reflecting recency more than retention.

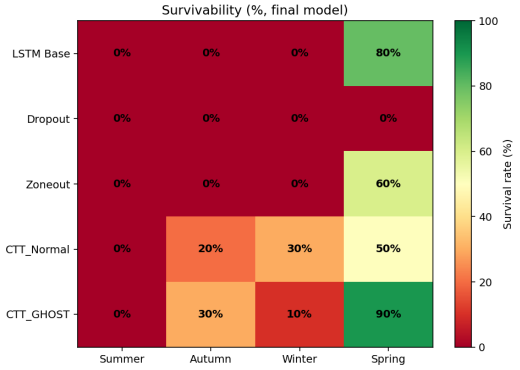


Figure 2: Heatmap of average survival of models over ten seeds

Further, CTT-architecture models survived significantly longer than baselines in mid-sequence seasons. In both autumn and winter, the Friedman test found significant differences among models (autumn $\chi^2 = 16.44$, $p = 0.003$; winter $\chi^2 = 22.92$, $p < 0.001$), and post-hoc comparisons show CTT models sustained significantly longer episodes than the LSTM baseline. In autumn, both CTT architectures exceeded the LSTM Base (Holm-corrected $p = 0.047$, rank-biserial 0.82-1.0). In winter, CTT_Normal ($p = 0.012$) and CTT_GHOST ($p = 0.047$) exceeded the LSTM Base. Figure 3 shows that baseline models typically expired at or near the starvation line (500 steps) in autumn and winter, whereas the CTT-architecture variants persisted for notably longer. This indicates that this architecture consolidates a measurable survival advantage over standard LSTMs even where neither approach achieves full survival.

Moreover, most omnibus results do not translate into pairwise differences. Note the significant Friedman results in spring across all metrics and summer across steps did not yield any Holm-significant pairwise comparisons (see Table 5 compared to Table 6 below). Through inspection, it indicates these omnibus are largely driven by Dropout, which, having failed to learn the task, performs uniformly poorly across all seasons, and inflates between-model variance without any specific pair differing after correction.

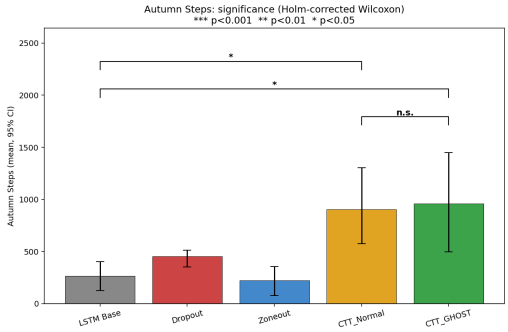
Noticeably, GHOST scoring did not improve on the original CTT scoring. Across all three metrics and all four seasons, no comparison between CTT_GHOST and CTT_Normal reached any statistical significance (i.e. all $p_{holm} > 0.19$,, see table 5 below). While CTT_GHOST showed numerically higher spring performance, these differences did not survive correction at $n = 10$. We therefore find no evidence that the Hebbian co-activation criterion notably outperforms the original redundancy/importance criterion. At best, they perform on par with one another. Thus, on evidence of final-model performance, the research question is answered negatively: Hebbian-scored turnover does not enable more robust consolidation than the original CTT scoring. The two are statistically indistinguishable across every metric and season.

Table 5: Friedman omnibus tests (significant seasons only).

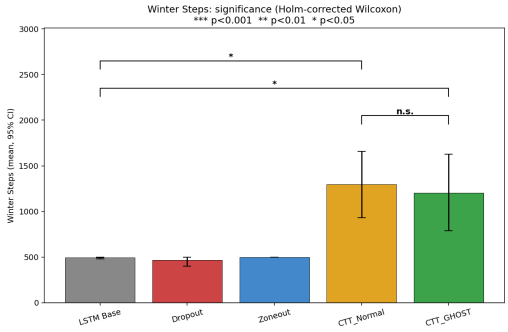
Metric	Season	χ^2 (p)
Reward	Spring	17.31 (0.002)
Surviv.	Spring	19.68 (< 0.001)
Steps	Summer	13.88 (0.008)
Steps	Autumn	16.44 (0.003)
Steps	Winter	22.92 (< 0.001)
Steps	Spring	26.69 (< 0.001)

Table 6: Pairwise p_{Holm}. G=GHOST, N=Normal, B=Base. Bold<0.05.

Metric	Season	G/N	G/B	N/B
Reward	Spring	0.48	0.55	0.55
Surviv.	Spring	0.375	1.00	0.75
Steps	Summer	0.50	0.50	0.094
Steps	Autumn	1.00	**0.047**	**0.047**
Steps	Winter	0.77	**0.047**	**0.012**
Steps	Spring	0.19	0.50	0.25



(a) Autumn



(b) Winter

Figure 3: Significance bars of step counts in the two mid-sequence seasons

4.3 Forgetting Analysis

The final-model results in the above section establish where performance lies, but not why it lies there: a season scored at the typical death/starvation rate may have been learned and then lost,

or not learned at all. To separate these, we compare each season's own checkpoints, its peak "learned" performance, against the final model's "remembered" performance on the same season. A one-sample gate test first confirms whether a season was learned, since forgetting is only defined for a season a model mastered.

This helps us move beyond Dropout, whose uniform failure made the results ambiguous in section 4.2. Dropout's peak performance never significantly exceeded the starvation floor in any season ($p \geq 0.12$), confirming it had never learned the task as opposed to learning and then forgetting it. Thus, dropout ought be excluded from any retention comparisons. The remaining four models did learn summer, autumn, and winter (gate $p < 0.05$; peak survivability 50-90%), so any collapse that follows is genuine forgetting.

That collapse is uniform for the earliest season. Every model that learned summer (peak survivability 80-90%, duration of 1795-2000 steps) regressed almost entirely to the starvation floor by the end of training (Remembered survivability of 0%, steps retention ≤ 0.14; paired Wilcoxon $p < 0.01$ for all). This is the quantifiable version of the survivability heatmap's suggestions (Figure 2): summer was demonstrably acquired and then almost entirely lost. While some seed evidence does point to the CTT-architectures remembering some summer strategies (see example Figures 4 and 5), it is not consistent enough to be statistically significant.

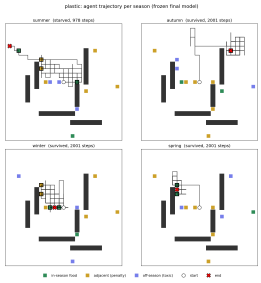


Figure 4: Path Plot of CTT_Normal on seed 21

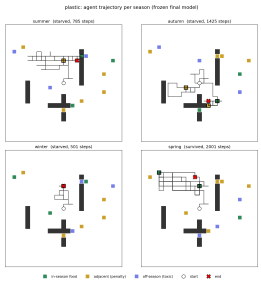


Figure 5: Path Plot of CTT_GHOST on seed 48

The turnover mechanism measurably slows this loss in the mid-sequence seasons. In autumn, both CTT architectures retained a meaningful fraction of their learned step-count (retention 0.30 and 0.35) and forgot significantly less than the LSTM Base (p_{Holm} = 0.023) whereas the baselines fell below the starvation floor (retention -0.16 and -0.21). This is to say that the forgotten baselines didn't merely starve to death, but began selecting toxic patches. Given that majority of these deaths happen in autumn, it suggests that the weights are too attenuated to spring, and thus in autumn evaluations, it tends towards spring patches, which are toxic. The winter result is more nuanced: while CTT_Normal forgot significantly less than the baseline (retention 0.73 v -0.01; $p_{Holm} = 0.047$), CTT_GHOST did not. Despite attaining significantly longer winter episodes in the final-model analysis (Section 4.2), its high final performance is offset by an equally high learned peak, leaving a large retention drop. CTT_Normal, with a lower peak but higher retention, is thus the only model to forget winter significantly less than the baseline.

Crucially, while both CTT_Normal and CTT_GHOST forget significantly less than the baseline in autumn, and one in winter, it does not suggest that this benefit is dependent on the scoring criterion. Rather, it suggests the overall architecture lends itself towards continual learning, with the scoring systems being on par with one another, if at all impacting the architecture's capacity to remember. Measured directly as retention, the drop from learned to remembered performance, the research question is again answered negatively: the Hebbian criterion did not forget significantly less than the original scoring in any season. Where reduced forgetting occurred, it was a property of the turnover architecture shared by both variants, not that of the scoring mechanism specifically.

4.4 Substantive Discussion Beyond The Results

Thus, taken across both final performance and direct retention analysis, the research question resolves negatively throughout: Hebbian-scored turnover does not produce more robust long or short term consolidation than scoring neurons through the original CTT criterion. Conclusively, hypothesis H1 is not supported as per our analysis above. Thus, the value of CTT in this setting lies in its turnover architecture, not necessarily in the criterion used to select which neurons survive. Both CTT architectures forget significantly less than the baseline in the mid-sequence seasons, confirming hypothesis H2 is supported, yet the two variants were statistically indistinguishable from one another on every metric and season (minimum $p_{Holm} = 0.19$). The Hebbian-inspired co-activation scoring mechanism that defines GHOST therefore confers no measurable advantage nor disadvantage against the original importance-redundancy criterion. Where there is resistance to catastrophic forgetting, it is assumed to be as a result of the turnover mechanism pipeline itself: structured death and rebirth of neurons with gating, as well as integrated EWC and replay consolidation mechanisms, rather than it being attributed to the criterion of the neurons selected. This is a useful negative result in that it localises the source of the benefit and suggests that efforts spent refining scoring functions is unlikely to be the most productive direction for improving turnover-based continual learning.

A plausible explanation is that neuron-level turnover protects which neurons exist in the network, but not necessarily the information the weights encode. A high-scoring neuron is spared from death, but its weights still remain fully trainable, and gradient descent under PPO continues to overwrite these weights during subsequent seasons (or tasks). Thus, the specific ranking produced

by a scorer is likely to be second-order: as long as turnover preserves a diverse enough population of plastic neurons, the exact identities it protects matters little, because forgetting is a weight update problem, not necessarily a neuronal problem. This interpretation is consistent with prior findings that biologically-motivated consolidation mechanisms which succeed for Hebbian-trained networks do not transfer to networks trained by gradient descent [34]. The agents were all trained by PPO, with the Hebbian-inspired signal being used only for selection, placing them in the regime where scoring-based protection alone would be insufficient, especially in such a difficult sequence of tasks, as we have observed.

No approach prevented the complete loss of the earliest season: every model that learned summer regressed to the starvation floor by the end of training. Moreover, the cases of poisoning, i.e. agents eating toxic out-of-season patches, suggest that forgetting is an active overwriting of policy towards recently trained seasons. The turnover and consolidation mechanisms in the CTT architecture were able to slow the process down in middle seasons, but still provided no protection to the first season. This suggests that neuron-level plasticity combined with EWC and replay, while beneficial, is not on its own a sufficient mechanism for continual learning in a difficult task set sequence within a gradient-trained recurrent agent. This suggests that the main obstacle doesn't necessarily lie in the consolidation mechanism, which literature shows is able to learn against catastrophic forgetting, but rather in the continual reinforcement-learning setting itself, where a shifting reward regime continually drives the policy toward the most recent season. This is especially important to note in that comparative to literature, this task set is doubly demanding: reinforcement learning generally yields weaker continual-learning results than supervised tasks, and most continual-learning experiments transition between distinct tasks, whereas here the environment is fixed and the reward function itself reverses between seasons. A patch that nourishes in one season becomes toxic or negative-rewarding in another. This is a severe case of non-stationarity, something acknowledged as a core difficulty of continual reinforcement learning (CRL) [33], and one most benchmarks avoid: few impose tasks that actively punish previously-correct behaviour.

These findings bear a broader implication for the design and frameworks of biologically-inspired continual-learning agents. The neuroscience that motivates architectures like CTT such as synaptic turnover, homeostasis, and consolidation, offer multiple candidate mechanisms, and it is enticing to assume a more principled criterion for which neurons to protect (as GHOST provides) should therefore improve retention. Our results caution against this assumption: a Hebbian-inspired criterion conferred no advantage over the original importance-redundancy scoring, and even the combination of turnover, EWC, and replay could not prevent the total loss of the earliest task. For practitioners, the practical guideline is that effort invested in refining the neuron *selection* criterion yields little return. The obstacle lies instead in the learning setting itself: the non-stationarity of the reward structure combined with the lack of any mechanism that protects what the surviving weights encode against continued gradient updates. This is not discouraging, however. This paper instead spotlights the locus of improvement: future design efforts in this setting must be redirected away from any sort of scoring sophistication and towards the reward structure, task-boundary handling, and weight-level protection that this adversarially non-stationary reinforcement learning setting demands.

5 Conclusions

This paper asks whether Hebbian-inspired scored neuron turnover during simulated sleep cycles enables more robust memory consolidation than the original CTT scoring, in an LSTM foraging agent trained across four sequential seasons. We find that it does not: the Hebbian criterion (GHOST) was statistically indistinguishable from the original redundancy-importance scoring on every metric and season. We do, however, find that turnover-based models retained significantly more of their learned behaviour than standard LSTM baselines in the mid-sequence seasons, locating the benefit of CTT as an architecture rather than in the choice of scoring function. No method prevented catastrophic forgetting of the earliest season, indicating that turnover in the plastic layer combined with EWC and replay is not necessarily sufficient to remember.

The contribution of this work acts as a caution for biologically-inspired CRL design: more sophisticated neuron-selection design does not translate into better retention, and even combined biological consolidation mechanisms fail under adversarial reward non-stationarity. For practitioners, this redirects design effort away from the scoring function, and more toward architectural and algorithmic design favoured for this continual learning task setting.

Two directions thus follow. Because the CTT framework combines several mechanisms at once, a systematic ablation isolating turnover, consolidation, and replay is needed to identify which, if any, drives the observed mid-season retention. More importantly, the failure of established consolidation methods in this specific task sequence setting suggests the central obstacle is not the choice of architectural consolidation mechanisms, but the adversarially non-stationary reinforcement-learning task itself. Understanding why methods effective in supervised continual learning do not transfer to reward-reversing reinforcement settings is, as we suggest, the more pressing open problem, and a prerequisite for any turnover-based method to succeed.

References

[1] Jimmy Ba and Brendan Frey. 2013. Adaptive Dropout for Training Deep Neural Networks. In *Advances in Neural Information Processing Systems (NIPS)*. 3084–3092. https://papers.nips.cc/paper/5032-adaptive-dropout-for-training-deep-neural-networks

[2] Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Rémi Munos. 2016. Unifying Count-Based Exploration and Intrinsic Motivation. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 29. https://proceedings.neurips.cc/paper_files/paper/2016/file/afda332245e2af431fb7b672a68b659d-Paper.pdf

[3] Metehan Cekic, Can Bakiskan, and Upamanyu Madhow. 2022. Towards Robust, Interpretable Neural Networks via Hebbian/anti-Hebbian Learning: A Software Framework for Training with Feature-Based Costs. *Software Impacts* 13 (2022), 100347. doi:10.1016/j.simpa.2022.100347

[4] Timothy J. Draelos, Nadine E. Miner, Christopher C. Lamb, Jonathan A. Cox, Craig M. Vineyard, Kristofor D. Carlson, William M. Severa, Conrad D. James, and James B. Aimone. 2017. Neurogenesis Deep Learning: Extending Deep Networks to Accommodate New Classes. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. doi:10.1109/IJCNN.2017.7965898

[5] Chrisantha Fernando, Dylan Banarse, Charles Blundell, Yori Zwols, David Ha, Andrei A. Rusu, Alexander Pritzel, and Daan Wierstra. 2017. PathNet: Evolution Channels Gradient Descent in Super Neural Networks. *arXiv preprint arXiv:1701.08734* (2017).

[6] Robert M. French. 1999. Catastrophic Forgetting in Connectionist Networks. *Trends in Cognitive Sciences* 3, 4 (1999), 128–135. doi:10.1016/S1364-6613(99)01294-2

[7] Mustafa Burak Gurbuz, Jean Michael Moorman, and Constantine Dovrolis. 2024. NICE: Neurogenesis Inspired Contextual Encoding for Replay-free Class Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 23659–23669. doi:10.1109/CVPR52733.2024.02233

[8] Donald O. Hebb. 1949. *The Organization of Behaviour*. Wiley, New York.

[9] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735

[10] Ferenc Huszár. 2018. Note on the Quadratic Penalties in Elastic Weight Consolidation. *Proceedings of the National Academy of Sciences* 115, 11 (2018), E2496–E2497. doi:10.1073/pnas.1717042115

[11] Adrien Journé, Hector Garcia Rodriguez, Qinghai Guo, and Timoleon Moraitis. 2023. Hebbian Deep Learning Without Feedback. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=8gd4M-_Rj1

[12] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences* 114, 13 (2017), 3521–3526. doi:10.1073/pnas.1611835114

[13] Chris Kortge. 1990. Episodic Memory in Connectionist Networks. In *Proceedings of the 12th Annual Conference of the Cognitive Science Society*. 764–771.

[14] David Krueger, Tegan Maharaj, János Kramár, Mohammad Pezeshki, Nicolas Ballas, Nan Rosemary Ke, Anirudh Goyal, Yoshua Bengio, Hugo Larochelle, Aaron C. Courville, and Chris Pal. 2017. Zoneout: Regularizing RNNs by Randomly Preserving Hidden Activations. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=rJqBEPcxe

[15] U.B. Mahadevaswamy and P. Swathi. 2023. Sentiment Analysis using Bidirectional LSTM Network. *Procedia Computer Science* 218 (2023), 45–56. doi:10.1016/j.procs.2022.12.400 International Conference on Machine Learning and Data Engineering.

[16] Robert Malenka. 2003. The Long-Term Potential of LTP. *Nature Reviews Neuroscience* 4 (2003), 923–926. doi:10.1038/nrn1258

[17] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. 1995. Why There Are Complementary Learning Systems in the Hippocampus and Neocortex. *Psychological Review* 102, 3 (1995), 419–457. doi:10.1037/0033-295X.102.3.419

[18] Michael McCloskey and Neal J. Cohen. 1989. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In *Psychology of Learning and Motivation*. Vol. 24. Elsevier, 109–165. doi:10.1016/S0079-7421(08)60536-8

[19] Randall C. O'Reilly. 1998. Six Principles for Biologically Based Computational Models of Cortical Cognition. *Trends in Cognitive Sciences* 2, 11 (1998), 455–462. doi:10.1016/S1364-6613(98)01241-8

[20] Kamil Rocki, Tomasz Kornuta, and Tegan Maharaj. 2016. Surprisal-Driven Zoneout. In *Workshop on Neural Abstract Machines and Program Induction, NIPS*.

[21] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Gregory Wayne. 2019. Experience Replay for Continual Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32.

[22] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[23] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[24] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 15, 1 (2014), 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

[25] Alexander L. Strehl and Michael L. Littman. 2008. An Analysis of Model-Based Interval Estimation for Markov Decision Processes. *J. Comput. System Sci.* 74, 8 (2008), 1309–1331. doi:10.1016/j.jcss.2007.08.009

[26] Rachael L. Sumner, Meg J. Spriggs, Suresh D. Muthukumaraswamy, and Ian J. Kirk. 2020. The Role of Hebbian Learning in Human Perception: A Methodological and Theoretical Review of the Human Visual Long-Term Potentiation Paradigm. *Neuroscience and Biobehavioral Reviews* 115 (2020), 220–237. doi:10.1016/j.neubiorev.2020.03.013

[27] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to Sequence Learning with Neural Networks. In *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger (Eds.), Vol. 27. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2014/file/5a18e133cbf9f257297f410bb7eca942-Paper.pdf

[28] Giulio Tononi and Chiara Cirelli. 2006. Sleep Function and Synaptic Homeostasis. *Sleep Medicine Reviews* 10, 1 (2006), 49–62. doi:10.1016/j.smrv.2005.05.002

[29] Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, et al. 2023. Gymnasium. Zenodo. doi:10.5281/zenodo.8127026

[30] Lina M. Tran, Adam Santoro, Lulu Liu, Sheena A. Josselyn, Blake A. Richards, and Paul W. Frankland. 2022. Adult Neurogenesis Acts as a Neural Regularizer. *Proceedings of the National Academy of Sciences* 119, 25 (2022). doi:10.1073/pnas.2206704119

[31] Gina G. Turrigiano, Kenneth R. Leslie, Niraj S. Desai, Lana C. Rutherford, and Sacha B. Nelson. 1998. Activity-Dependent Scaling of Quantal Amplitude in Neocortical Neurons. *Nature* 391, 6670 (1998), 892–896. doi:10.1038/36103

[32] Sida Wang and Christopher Manning. 2013. Fast Dropout Training. In *Proceedings of the International Conference on Machine Learning (ICML)*. 118–126. https://proceedings.mlr.press/v28/wang13a.html

[33] Maciej Wołczyk, Michał Zając, Razvan Pascanu, Łukasz Kuciński, and Piotr Miłoś. 2021. Continual World: A Robotic Benchmark for Continual Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 28496–28510. https://proceedings.neurips.cc/paper_files/paper/2021/file/ef8446f35513a8d6aa2308357a268a7e-Paper.pdf

[34] Tianyi Xu, Patrick Zheng, Shiyan Liu, Sicheng Lyu, and Isabeau Prémont-Schwarz. 2024. SNAP: Stopping Catastrophic Forgetting in Hebbian Learning with Sigmoidal Neuronal Adaptive Plasticity. In *Montréal AI and Neuroscience (MAIN) Conference*.

[35] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. 2018. Lifelong Learning with Dynamically Expandable Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=Sk7KsfW0-

A AI Usage Statement

This paper made use of Claude Sonnet 5 to check for any spelling and grammatical mistakes (mainly possessive forms). With LaTeX, Claude had also assisted with formatting figures only, which was copy-pasted, and then edited to specifically fit my paper (e.g. doing subgraphs as with Figure 3). Similarly, all tables in the Appendix sections C and D were formatted by Claude Sonnet 5, but using the real data which can be found in my repo linked in footnote. No words, or any material analysis found in this paper, were written by Claude or any Gen-AI (or otherwise) models. All analysis was formulated by myself, and all words were written by myself.

B Seeds

Seeds used were:

(1) 27
(2) 7
(3) 21
(4) 19
(5) 16
(6) 99
(7) 66
(8) 48
(9) 84
(10) 73

C Full Statistical Results

Table 7: Descriptive statistics for *Reward* (mean episodic reward). B = Base, Dr = Dropout, Zo = Zoneout, N = CTT_Normal, G = CTT_GHOST.

Model	Median	Mean	95 % CI
Summer — Friedman $\chi^2 = 4.47$, $p = 0.346$			
B	−140.38	−144.19	[−156.26, −133.47]
Dr	−159.00	−156.95	[−170.66, −143.26]
Zo	−137.55	−137.22	[−142.22, −132.34]
N	−147.80	−152.39	[−163.24, −142.43]
G	−140.30	−140.68	[−149.69, −132.74]
Autumn — Friedman $\chi^2 = 6.31$, $p = 0.177$			
B	−110.65	−119.13	[−132.99, −106.25]
Dr	−160.42	−151.24	[−167.93, −133.59]
Zo	−103.68	−110.77	[−118.25, −103.32]
N	−146.91	−111.38	[−164.03, −40.74]
G	−117.60	−86.20	[−145.80, −14.52]
Winter — Friedman $\chi^2 = 5.31$, $p = 0.257$			
B	−140.82	−141.93	[−151.20, −133.73]
Dr	−157.68	−151.34	[−165.50, −137.66]
Zo	−137.55	−138.21	[−142.34, −134.18]
N	−131.47	−65.64	[−129.80, 10.87]
G	−127.86	−109.70	[−139.88, −69.97]
Spring — Friedman $\chi^2 = 17.31$, $p = 0.002$ (sig.)			
B	90.59	66.34	[−12.64, 136.26]
Dr	−160.54	−152.91	[−168.15, −136.97]
Zo	43.03	34.89	[−33.99, 105.40]
N	−41.95	−4.63	[−92.57, 86.42]
G	151.25	102.08	[34.01, 160.21]

Table 8: Holm-corrected pairwise Wilcoxon comparisons for *Reward*. Bold $p_{\text{Holm}} < 0.05$.

Season	Comparison	p	p_{Holm}	r_{rb}
Summer	G vs N	0.160	0.481	+0.527
	G vs B	1.000	1.000	−0.018
	N vs B	0.492	0.984	−0.273
Autumn	G vs N	1.000	1.000	−0.018
	G vs B	0.652	1.000	+0.200
	N vs B	0.846	1.000	−0.091
Winter	G vs N	0.846	0.846	−0.091
	G vs B	0.084	0.252	+0.636
	N vs B	0.193	0.387	+0.491
Spring	G vs N	0.160	0.481	+0.527
	G vs B	0.492	0.551	+0.273
	N vs B	0.275	0.551	−0.418

Table 9: Descriptive statistics for *Survivability* (% of episodes reaching the 2000-step limit). Floor % = fraction of seeds at or below the 0.05 collapse threshold.

Model	Median	Mean	95 % CI	Floor %
Summer — Friedman undefined (all at floor)				
B	0	0	[0, 0]	100
Dr	0	0	[0, 0]	100
Zo	0	0	[0, 0]	100
N	0	0	[0, 0]	100
G	0	0	[0, 0]	100
Autumn — Friedman $\chi^2 = 8.89$, $p = 0.064$				
B	0	0	[0, 0]	100
Dr	0	0	[0, 0]	100
Zo	0	0	[0, 0]	100
N	0	20	[0, 50]	80
G	0	30	[0, 60]	70
Winter — Friedman $\chi^2 = 8.50$, $p = 0.075$				
B	0	0	[0, 0]	100
Dr	0	0	[0, 0]	100
Zo	0	0	[0, 0]	100
N	0	30	[0, 60]	70
G	0	10	[0, 30]	90
Spring — Friedman $\chi^2 = 19.68$, $p < 0.001$ (sig.)				
B	100	80	[50, 100]	20
Dr	0	0	[0, 0]	100
Zo	100	60	[30, 90]	40
N	50	50	[20, 80]	50
G	100	90	[70, 100]	10

Table 10: Holm-corrected pairwise Wilcoxon comparisons for *Survivability*. Bold $p_{\text{Holm}} < 0.05$.

Season	Comparison	p	p_{Holm}	r_{rb}
Summer	G vs N	1.000	1.000	+0.000
	G vs B	1.000	1.000	+0.000
	N vs B	1.000	1.000	+0.000
Autumn	G vs N	1.000	1.000	+0.333
	G vs B	0.250	0.750	+1.000
	N vs B	0.500	1.000	+1.000
Winter	G vs N	0.625	1.000	−0.500
	G vs B	1.000	1.000	+1.000
	N vs B	0.250	0.750	+1.000
Spring	G vs N	0.125	0.375	+1.000
	G vs B	1.000	1.000	+0.333
	N vs B	0.375	0.750	−0.600

Table 11: Descriptive statistics for *Steps* (median episode duration; ≈500 = starvation, 2000 = full survival).

Model	Median	Mean	95 % CI
Summer — Friedman $\chi^2 = 13.88$, $p = 0.008$ (sig.)			
B	501.00	501.00	[501.00, 501.00]
Dr	501.00	508.20	[501.00, 518.70]
Zo	501.00	501.00	[501.00, 501.00]
N	562.00	709.60	[552.00, 893.50]
G	501.00	550.60	[501.00, 611.60]
Autumn — Friedman $\chi^2 = 16.44$, $p = 0.003$ (sig.)			
B	283.50	267.40	[126.50, 403.80]
Dr	501.00	455.40	[352.50, 512.70]
Zo	91.50	223.10	[78.59, 368.20]
N	527.50	905.40	[573.70, 1303.91]
G	566.00	959.10	[493.17, 1448.80]
Winter — Friedman $\chi^2 = 22.92$, $p < 0.001$ (sig.)			
B	501.00	494.80	[482.40, 501.00]
Dr	501.00	466.90	[398.70, 501.00]
Zo	501.00	501.00	[501.00, 501.00]
N	1274.50	1298.20	[925.80, 1666.20]
G	1267.00	1203.70	[784.10, 1614.82]
Spring — Friedman $\chi^2 = 26.69$, $p < 0.001$ (sig.)			
B	2001.00	1796.90	[1496.90, 2001.00]
Dr	501.00	490.00	[455.30, 512.70]
Zo	2001.00	1934.80	[1864.60, 2001.00]
N	1713.00	1416.10	[1009.50, 1793.40]
G	2001.00	1974.50	[1921.50, 2001.00]

Table 12: Holm-corrected pairwise Wilcoxon comparisons for *Steps*. Bold $p_{\text{Holm}} < 0.05$.

Season	Comparison	p	p_{Holm}	r_{rb}
Summer	G vs N	0.301	0.500	−0.422
	G vs B	0.250	0.500	+1.000
	N vs B	0.031	0.094	+1.000
Autumn	G vs N	1.000	1.000	+0.000
	G vs B	0.020	**0.047**	+0.818
	N vs B	0.016	**0.047**	+1.000
Winter	G vs N	0.770	0.770	−0.127
	G vs B	0.023	**0.047**	+0.889
	N vs B	0.004	**0.012**	+1.000
Spring	G vs N	0.062	0.188	+1.000
	G vs B	0.500	0.500	+0.667
	N vs B	0.125	0.250	−0.867

D Full Forgetting Analysis Results

Table 13: Pairwise forgetting comparisons for *Survivability* (Holm-corrected Wilcoxon; lower forgetting is better). Spring is excluded as the no-forgetting control. Bold $p_{\text{Holm}} < 0.05$.

Season	Comparison	p	p_{Holm}
Summer	G vs N	1.000	1.000
	G vs B	1.000	1.000
	N vs B	1.000	1.000
Autumn	G vs N	0.625	0.750
	G vs B	0.062	0.188
	N vs B	0.375	0.750
Winter	G vs N	0.750	1.000
	G vs B	1.000	1.000
	N vs B	0.500	1.000

Table 14: Pairwise forgetting comparisons for *Steps* (Holm-corrected Wilcoxon; lower forgetting is better). Spring is excluded as the no-forgetting control. Bold $p_{\text{Holm}} < 0.05$.

Season	Comparison	p	p_{Holm}
Summer	G vs N	0.820	0.820
	G vs B	0.125	0.250
	N vs B	0.031	0.094
Autumn	G vs N	0.914	0.914
	G vs B	0.010	**0.023**
	N vs B	0.008	**0.023**
Winter	G vs N	0.447	0.447
	G vs B	0.131	0.262
	N vs B	0.016	**0.047**

Table 15: Forgetting analysis for *Survivability* (floor = 0). Friedman is computed on the *Forget* values across all five models. * $p < 0.05$; † season not learned (gate $p \geq 0.05$), forgetting not interpretable; n/a = Retain undefined (Learned = 0).

Model	Learned	Remem.	Forget	Retain	Gate p	Forgot p
Summer — Friedman $\chi^2 = 24.40$, $p < 0.001$ (sig.)						
B	90.0	0.0	90.0	0.00	0.002	0.004*
Dr†	10.0	0.0	10.0	0.00	0.500	1.000
Zo	80.0	0.0	80.0	0.00	0.004	0.008*
N	90.0	0.0	90.0	0.00	0.002	0.004*
G	90.0	0.0	90.0	0.00	0.002	0.004*
Autumn — Friedman $\chi^2 = 17.33$, $p = 0.002$ (sig.)						
B	90.0	0.0	90.0	0.00	0.002	0.004*
Dr†	0.0	0.0	0.0	n/a	1.000	1.000
Zo	70.0	0.0	70.0	0.00	0.008	0.016*
N	80.0	20.0	60.0	0.25	0.004	0.031*
G	70.0	30.0	40.0	0.43	0.008	0.125
Winter — Friedman $\chi^2 = 7.89$, $p = 0.096$ (n.s.)						
B	50.0	0.0	50.0	0.00	0.031	0.062
Dr†	0.0	0.0	0.0	n/a	1.000	1.000
Zo	50.0	0.0	50.0	0.00	0.031	0.062
N	50.0	30.0	20.0	0.60	0.031	0.625
G	50.0	10.0	40.0	0.20	0.031	0.125
Spring — no-forgetting control (checkpoint = final model)						
B	80.0	80.0	0.0	1.00	0.004	—
Dr†	0.0	0.0	0.0	n/a	1.000	—
Zo	60.0	60.0	0.0	1.00	0.016	—
N	50.0	50.0	0.0	1.00	0.031	—
G	90.0	90.0	0.0	1.00	0.002	—

Table 16: Forgetting analysis for *Steps* (floor = 500). Columns and markers as in Table 15. Negative Retain indicates the final model fell below the starvation floor (toxic-patch deaths), i.e. worse than never eating.

Model	Learned	Remem.	Forget	Retain	Gate p	Forgot p
Summer — Friedman $\chi^2 = 23.41$, $p < 0.001$ (sig.)						
B	1999.1	500.0	1499.1	0.00	0.001	0.002*
Dr†	653.9	507.4	146.5	0.05	0.250	0.875
Zo	1795.7	500.0	1295.7	0.00	0.001	0.002*
N	1960.3	709.2	1251.1	0.14	0.001	0.002*
G	1850.0	549.9	1300.1	0.04	0.002	0.004*
Autumn — Friedman $\chi^2 = 20.64$, $p < 0.001$ (sig.)						
B	1999.5	267.0	1732.5	−0.16	0.001	0.002*
Dr†	536.2	454.6	81.6	−1.25	0.125	0.375
Zo	1792.3	222.7	1569.6	−0.21	0.001	0.002*
N	1847.4	904.7	942.7	0.30	0.002	0.016*
G	1793.2	958.5	834.7	0.35	0.002	0.031*
Winter — Friedman $\chi^2 = 9.98$, $p = 0.041$ (sig.)						
B	1598.2	493.9	1104.3	−0.01	0.004	0.008*
Dr†	622.0	466.0	156.0	−0.28	0.125	0.250
Zo	1445.9	500.0	945.9	0.00	0.008	0.016*
N	1588.3	1297.8	290.5	0.73	0.002	0.156
G	1763.4	1203.3	560.1	0.56	0.001	0.004*
Spring — no-forgetting control (checkpoint = final model)						
B	1795.9	1796.0	−0.1	1.00	0.002	—
Dr†	489.0	489.3	−0.3	n/a	0.625	—
Zo	1783.8	1934.1	−150.3	1.12	0.002	—
N	1415.1	1415.4	−0.3	1.00	0.004	—
G	1973.5	1973.6	−0.1	1.00	0.001	—